技术文档：基于大模型的多模态视觉理解与推理

对应提交文件：AIC-2026-14375345-赛题结果文档/result.zip（内部文件名 submission.json）

1. 任务与方案概述

1.1 任务定义

赛题要求给定同一场景的**可见光（visible）、红外（infrared）、深度（depth）**三模态图像，以及一条描述目标的英文查询，输出该目标在可见光图像中的**归一化边界框** [x1, y1, x2, y2]（x1/y1 为左上角、x2/y2 为右下角，均由像素坐标除以图像宽高得到，取值范围 0~1）。

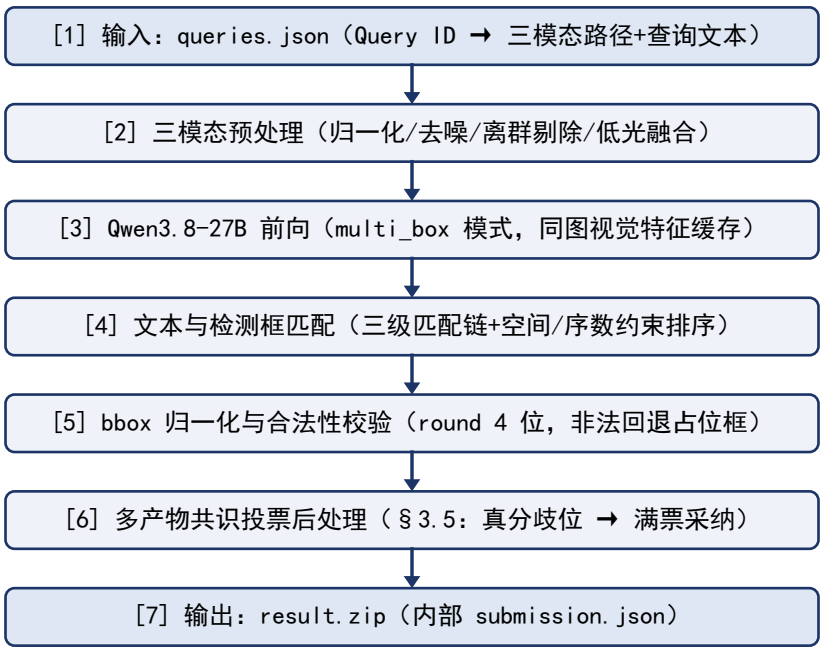
评测指标为 **ACC@0.5**：预测框与真实框的交并比 IoU ⩾ 0.5 记为该条正确，最终得分为正确条数占总查询数的比例。

1.2 方案路线

本方案采用**预训练多模态大语言模型 Qwen3.8-27B-Instruct 的零样本推理**路线。选择该路线的原因有三：

- **任务适配**：查询文本常含外观描述、空间关系、序数指代（"从右往左第三个"）等需要语义推理的成分，生成式大模型在语言-视觉对齐与常识推理上具备优势；
- **合规友好**：赛题要求仅使用开源模型，且禁止调用闭源模型在线 API——本地部署的 Qwen3.8-27B 满足要求；
- **无需训练**：赛题不提供训练集，零样本路线避免了在无监督信号下做适配带来的过拟合风险。

1.3 整体流程



其中第 1–5 步由 scripts/infer.py 端到端完成，第 6 步由 scripts/consensus_vote.py 独立完成，第 7 步为打包输出；两步之间以预测 JSON 为接口，可分别复现与校验。

1.4 赛题合规要点

- **模型**：开源模型 Qwen3.8-27B，本地部署推理，全程无商业闭源 API 调用；
- **数据**：仅使用赛事官方测试集，未使用外部数据、未对测试集做任何标注；
- **推理**：结果全部由算法推理产出，不含任何人工指定；后处理为确定性规则，可由管线逐条复算。

2. 数据集

本方案仅使用赛事官方提供的测试集进行推理，未使用任何外部训练数据（零样本推理，无训练环节）。

项目	内容
图像规模	1295 张场景图像（三模态各一份）
查询规模	5690 条查询（多查询/图）
模态	visible/infrared/depth（depth 为深度图）
输入	三模态图像路径+queries.json
输出	Query ID → {visible, infrared, depth, query, bbox}

三模态数据特性及其对方案的约束：

- **可见光**：三通道 8 位 RGB，提供外观纹理与色彩，是主要定位依据，但弱光场景下细节损失严重；
- **红外**：实质为单通道热辐射灰度图的三通道堆叠，像素值越高表示温度越高。在可见光失效的弱光场景中，红外可突显活体与热源轮廓，是主要的补充模态；
- **深度**：单通道 16 位，单位为毫米，相机可感知距离约 300~19999 mm（编码取值范围 0~19999）；数值为 0 或过小表示无效区域。深度提供目标与相机的空间距离，可用于区分"近处/远处"这类空间指代。

3. 技术方案

3.1 模型

属性	取值
模型	Qwen3.8-27B-Instruct（ModelScope Qwen/Qwen3.8-27B）
参数量	27.78B
精度	bfloat16
权重	Safetensors 18 分片，约 54GB
加载	AutoModelForImageTextToText+AutoProcessor

模型以 transformers 原生接口加载与推理，未做任何微调或参数修改。选择 27B 规模的动因在于：本赛题的困难样本集中在遮挡、同类物并存、序数指代等需要细粒度语义区分的场景，参数量更大的模型在开放词汇定位上的零样本表现更稳定。

3.2 多模态输入与预处理（src/data/）

预处理针对三模态各自的数据特性分别设计，目的是让模型接收到的输入在量纲、噪声与有效区域上保持一致：

- **可见光**：uint8 → float32 / 255.0 归一化到 [0,1]，统一量纲以避免与模型内部归一化叠加；
- **红外**：三通道实为同一灰度辐射图的三次堆叠，先转灰度、再中值滤波去除椒盐噪声、最后复制为三通道伪 RGB，消除通道冗余并抑制传感器噪点；
- **深度**：超出编码取值范围（0~19999 毫米）的像素判定为无效区并置零，再线性归一化并保留有效区掩码（详见§3.3.4），避免无效值被模型误读为"极近"或"极远"；

- **低光融合路径**：当可见光平均亮度低于阈值（low_light_threshold: 60）时判定为弱光场景，工程内提供两条增强路径——fuse_rgb_infrared（红外按固定权重与可见光加权，配合直方图均衡）与 gated_fusion（逐像素 Sigmoid 门控，深度无效区硬置 0）；赛题背景中"夜间可见光模糊时由红外突显目标热源轮廓"即指此类情形；
- **输入组织**：三模态以分图（split）方式送入视觉编码器（multimodal_input_mode: "split"），保持各自的空间对齐关系，不做通道拼接，以避免不同物理量纲的图像被混合编码。可见光原样输入；红外与深度分别经去噪与掩码处理后作为独立模态输入，三图按「可见光 → 红外 → 深度」固定顺序送入，提示词附「Image 1 为可见光、Image 2 为红外热像、Image 3 为深度图」的角色说明，并要求 bbox 一律给出在 Image 1（可见光）坐标系上。

3.3 推理（src/inference/、src/models/）

推理把「语义理解」与「坐标计算」分开：模型负责识别与定位，凡能用纯几何规则确定性表达的部分（序数、方位、相对参照）都交给规则层，避免让模型做它最不擅长的计数与排序。

3.3.1 生成与缓存

- **生成模式**：multi_box —— 同一张图像的多条查询一次前向生成多个检测框，相比"逐查询单框"显著减少前向次数；生成上限 max_new_tokens: 1024；
- **视觉特征缓存**：以图像内容哈希为键的有界 LRU（cache_max_entries: 4）。同图多查询场景下视觉塔仅编码一次，后续查询复用视觉特征——这是 multi_box 模式下最有效的加速手段，直接决定全量推理的可行时长；

3.3.2 候选匹配与序数层

复赛中约 **19.4%** 的查询含序数词（first/second/third … from left/right/top/bottom）。通用 VLM 能定位到**某个同类物体**，但未必是查询**指定的那一个**——让它做"从右数第三个"这类计数与排序，正是它最不擅长的部分。据此把该步拆成两半：

候选匹配链

模型输出的坐标经容错解析后，由 matcher 执行三级匹配链——① 精确标签匹配 → ② 双向单词子集软匹配（容忍模型回显多/少若干修饰词）→ ③ 顺序子串匹配（容忍词序插入）；命中候选多于一个且查询含空间或序数约束时，进入下述序数选择，无约束则取置信度最高者；

```
# src/inference/matcher.py —— 三级匹配链 + 按约束排序取第 k 个
def match_by_position(query: str, detections: list[dict]) -> dict | None:
    """三级匹配链：精确 -> 双向子集 -> 顺序子串 -> 空间排序 -> 最高分。"""
    q_norm = normalize_text(query)

    cands = [d for d in detections if normalize_text(d["label"]) == q_norm]

    if not cands:
        cands = [
            d for d in detections
            if _word_level_substring(d["label"], query)
            or _word_level_substring(query, d["label"])
        ]

    if not cands:
        cands = [
            d for d in detections
            if _sequential_substring(d["label"], query)
            or _sequential_substring(query, d["label"])
        ]

    if not cands:
        return None

    cons = parse_spatial_constraint(query)
```

```
    if cons and cons.get("kind") == "depth":
        return max(cands, key=lambda d: d["score"])

    if cons and len(cands) > 1:
        axis = 0 if cons["axis"] == "x" else 1

        def key(d):
            box = d["bbox_pixel"]
            cx = (box[0] + box[2]) / 2
            cy = (box[1] + box[3]) / 2
            return cx if axis == 0 else cy

        cands.sort(key=key, reverse=(cons["direction"] < 0))
        rank = min(max(cons["rank"], 1), len(cands))
        return cands[rank - 1]

    return max(cands, key=lambda d: d["score"])
```

提示词层（qwen3_vl_parse.py 规则 1）

要求模型从查询指定的方向边缘**起算**，逐个计数匹配物体，**只框第 n 个**；明确禁止"取最大/最显著目标"，并禁止把相邻物体一并框入——把"数"的负担降到最低；

```
# src/models/qwen3_vl_parse.py —— 定位提示词（规则 1：序数扫描；规则 2：参照系）
def build_query_prompt(query: str, modality_labels: Sequence[str] | None = None) -> str:
    """模式①：逐查询单框定位指令（要求 [0,1000] 的 JSON）。

    ``modality_labels`` 非空时前置「多图角色说明」（多模态多图输入）。
    """
    return (
        _modality_preamble(modality_labels)
        + "Locate the single target described below in the image. "
        'Return ONLY a JSON object of the form {"bbox_2d": [x1, y1, x2, y2]} '
        "where each coordinate is an integer in the normalized range [0, 1000]. "
        "Rules: "
"(1) When a query says 'the first/second/third/leftmost/rightmost' object, START SCANNING from the specified
            direction edge (left edge for left-to-right), count each matching object one by one in order, and box
            ONLY the nth one. Do NOT pick the largest or most prominent object; pick the one at the exact ordinal
            position. Do not include adjacent objects. "
"(2) Determine left/right reference from the query: if it says 'X's left/right' or 'left/right of X', use X's own
            left/right (if X faces you, X's right is on your left); if it says 'on the left/right of the image/scene'
            or 'leftmost/rightmost in the frame', use the image's left/right. "
"(3) When a query asks for a part of an object (headlight, logo, label, hand, mirror, wheel), box ONLY that small
            part, not the whole object. "
        "(4) Box tightly around the target with minimal background. "
"(5) IF the query mentions a color word (white/red/black/green/blue/yellow/orange/etc.), that color is a HARD
            constraint: do NOT box an object of the wrong color even if its position seems right. Keep scanning until
            you find an object matching BOTH the position AND the color. If no color is mentioned in the query,
            ignore this rule. "
        "Do not output any other text.\n"
        f"Target description: {clean_prompt_text(query)}"
    )
```

几何层（matcher.parse_spatial_constraint）

从查询解析出 (轴, 方向, rank)，对候选框按该轴中心坐标排序后取第 rank 个。"从某端数第 k 个"由此变成一次确定性排序，结果不再依赖模型是否"数对"；

```
# src/inference/matcher.py —— 方位与序数约束解析（与"排序取第 k 个"同口径）
def parse_spatial_constraint(query: str) -> dict | None:
    """解析空间/序数约束。
```

```
    支持：first/second/... from the left/right/top/bottom;
    from left to right, the first ...; leftmost/rightmost/topmost/bottommost;
    closest/nearest/farthest/furthest（kind="depth"）。
    """
    q = normalize_text(query)

    # 黑名单必须**词边界**匹配。原先用朴素子串 `f"first {blocked}" in q`，
    # 会把 "first handrail from the left" 里的 "first hand" 当黑名单删掉，
    # 剩 "rail from the left" → 序数丢失、约束静默降级为 None。
    for blocked in _FIRST_BLACKLIST:
        q = re.sub(rf"\bfirst\s+{blocked}\b", "", q)

    ord_pat = "|".join(_ORDINALS)

    # 桥（序数与 from 之间的定语）先按原式匹配；原式只允许「纯单词 + 空白」，
    # 于是 "first white diamond-shaped sign from bottom to top" 这类带连字符 /
    # 逗号 / 撇号的定语整条掉出解析（实测 22 条）。
    m = re.search(
        rf"\b({ord_pat})\b\s+(?:\w+\s+)*?from\s+(?:the\s+)?(left|right|top|bottom)",
        q,
    )
    # 放宽桥：仅在原式未命中时启用，故对原本能解析的 query **逐一等价**
    #  （group(1)/group(2) 是同样的词，返回的 axis/direction/rank 完全相同），
    # 对全量是严格超集——不可能让任何已解析的 query 失败或改判。
    if not m:
        m = re.search(
            rf"\b({ord_pat})\b[\s\w,.'\-]{{0,90}}?from\s+(?:the\s+)?"
            r"(left|right|top|bottom)",
            q,
        )
    if m:
        return {
            "axis": "x" if m.group(2) in ("left", "right") else "y",
            "direction": 1 if m.group(2) in ("left", "top") else -1,
            "rank": _ORDINALS[m.group(1)],
        }

    m = re.search(
        rf"from\s+(left|right)\s+to\s+(right|left)[,.]?\s+(?:the\s+)?({ord_pat})\b",
        q,
    )
    if m:
        return {
            "axis": "x",
            "direction": 1 if m.group(1) == "left" else -1,
            "rank": _ORDINALS[m.group(3)],
        }

    m = re.search(r"\b(leftmost|rightmost|topmost|bottommost)\b", q)
    if m:
        return {
            "axis": "x" if m.group(1) in ("leftmost", "rightmost") else "y",
            "direction": 1 if m.group(1) in ("leftmost", "topmost") else -1,
            "rank": 1,
        }

    m = re.search(r"\b(closest|nearest|farthest|furthest)\b", q)
    if m:
        is_near = m.group(1) in ("closest", "nearest")
        return {
            "axis": "y",
```

```
            "direction": 1 if is_near else -1,
            "rank": 1,
            "kind": "depth",
        }

    return None
```

枚举—选择显式化（detect_select.py）

把上述两步显式化为"枚举+选择"——先让模型 List EVERY <NP> 逐个给出同类实例（不做序数判断），再按 x/y 中心排序取第 k 个；closest/farthest 类改用深度图 ROI 平均深度排序，**深度不可用时宁可回退常规分支也不猜**。其约束解析额外覆盖 from left to right, the first …、<ord> lowest/highest floor、from near to far、in the middle 四类语序变体；

```
# src/inference/detect_select.py —— 确定性选择：按轴排序取第 k 个
def box_centers(boxes: Sequence[Sequence[float]], axis: str) -> List[float]:
    """一组 xyxy 框在给定轴上的中心坐标（``axis`` ∈ {"x", "y"}）。"""
    idx = 0 if axis == "x" else 1
    return [(float(b[idx]) + float(b[idx + 2])) / 2.0 for b in boxes]


def select_index(values: Sequence[float], constraint: Optional[dict]) -> Optional[int]:
    """按约束在 ``values`` 里选出目标下标（第 2 步）。

    ``values`` 对 ``kind="pos"`` 是框中心坐标，对 ``kind="depth"`` 是 ROI 平均深度
    （越小越近）。方向语义：``direction=+1`` 时 rank 1 取最小值，``-1`` 取最大值。
    越界 rank 夹到端点；``middle`` 取排序后的中间项。
    """
    n = len(values)
    if n == 0 or not constraint:
        return None
    if constraint.get("kind") == "middle":
        order = sorted(range(n), key=lambda i: values[i])
        return order[len(order) // 2]
    rank = constraint.get("rank")
    if rank is None:
        return None
    order = sorted(range(n), key=lambda i: values[i],
                   reverse=(int(constraint.get("direction", 1)) < 0))
    return order[min(max(int(rank), 1), n) - 1]
```

跨查询一致性（chains.py）

同图、同物体（残余实义词相同）、同轴同向的序数链，其框中心必须随序数单调——实测 181 条有效链中有 30% 违反该约束。对破序链做**排序式重指派**：同链成员为同类物体、框可互换，按位置排序后依序数重新指派即 L1-最优分配（另提供 PAVA 保序回归变体）；已单调的链完全不动，因此该操作幂等。

```
# src/inference/chains.py —— 序数链的构成与「排序式重指派」
# 另有 pava_non_decreasing()（PAVA 保序回归，供 mode="project" 使用）与
# classify_chain()（链状态审计）此处从略。
def chain_of(query: str) -> Optional[dict]:
    """从单条 query 解析出链成员信息；非序数链返回 None。

    Returns:
        ``{"axis": "x"|"y", "direction": 1|-1, "rank": int, "key": frozenset}``
         （direction=1 表示 rank 越大坐标越大，如 "first ... from the left"）。
    """
    cons = parse_spatial_constraint(query)
    if not cons or cons.get("kind") == "depth" or int(cons.get("rank", 0)) <= 0:
        return None
    key = residual_tokens(query)
    if not key:
        return None
```

```
    return {
        "axis": cons["axis"],
        "direction": int(cons["direction"]),
        "rank": int(cons["rank"]),
        "key": key,
    }

def enforce_ordinal_chains(
    queries: Sequence[str],
    boxes: Sequence[Optional[Sequence[float]]],
    mode: str = "sort",
    gap_ratio: float = 0.0,
    max_shift: Optional[float] = None,
) -> List[Optional[List[float]]]:
    """对同一场景内的 query 做序数链单调性修正（排序式重指派）。

    只处理同一次调用内（即同一张图）的 query：链的 key = (轴向, 方向, 残余词集合)，
    要求成员数 >= 2 且 rank 互不相同（同 rank 的重复 query 视为同一目标，取均值
    后一起平移）。

    Args:
        queries: 同图 query 文本（与 ``boxes`` 等长、同序）。
        boxes: 归一化 xyxy 框，缺失用 None 表示（None 不参与、原样返回）。
        mode: ``"sort"`` = 按位置排序后重指派（默认，L1 最优的序数分配）；
            ``"project"`` = PAVA 保序回归（只做最小 L2 微调，不改变指派）。
        gap_ratio: 相邻序数的最小中心间隔 = gap_ratio  链内平均框宽（默认 0.0，
            即已单调的链完全不动；>0 时会把并列/重合的链推开）。
        max_shift: 单框最大平移量（归一化坐标，None = 不限）。

    Returns:
        与 ``boxes`` 等长的新列表（未参与修正的项原样返回）。
    """
    out: List[Optional[List[float]]] = [
        None if b is None else [float(v) for v in b[:4]] for b in boxes
    ]

    groups: Dict[tuple, List[Tuple[int, int]]] = {}
    for i, q in enumerate(queries):
        if out[i] is None:
            continue
        ch = chain_of(q)
        if ch is None:
            continue
        groups.setdefault((ch["axis"], ch["direction"], ch["key"]), []).append(
            (ch["rank"], i)
        )

    for (axis, direction, _key), members in groups.items():
        ranks = sorted({r for r, _ in members})
        if len(ranks) < 2:
            continue
        axis_i = 0 if axis == "x" else 1

        rank_idx = {r: [i for rr, i in members if rr == r] for r in ranks}
        centers = [
            float(np.mean([_center(out[i], axis_i) for i in rank_idx[r]]))
            for r in ranks
        ]
        extents = [
            abs(float(out[i][2]) - float(out[i][0])) if axis_i == 0
            else abs(float(out[i][3]) - float(out[i][1]))
```

```
            for _, i in members
        ]
        gap = max(0.0, float(gap_ratio)) * (float(np.mean(extents)) if extents else 0.0)

        sign = 1.0 if direction > 0 else -1.0
        s = [sign * c for c in centers]
        new_s = pava_non_decreasing(s) if mode == "project" else sorted(s)

        if gap > 0:
            for k in range(1, len(new_s)):
                if new_s[k] < new_s[k - 1] + gap:
                    new_s[k] = new_s[k - 1] + gap

        for k, r in enumerate(ranks):
            delta = sign * new_s[k] - centers[k]
            if max_shift is not None:
                delta = max(-max_shift, min(max_shift, delta))
            if abs(delta) <= 1e-12:
                continue
            for i in rank_idx[r]:
                out[i] = _shift_box(out[i], delta, axis_i)

    return out
```

以上均为**零样本**手段：只调整提示词与后处理规则，不训练、不伪标注、不使用任何真值信息。

3.3.3 相对位置与参照物语义

查询中的方位词存在两种参照系，若不区分就会系统性取错：

- **以物体自身为参照**：the right side of the car 指车自身的右侧——当车面向镜头时，车的右侧落在**画面**左侧；
- **以图像为参照**：on the right of the image、rightmost in the frame 指**画面**右缘。

提示词规则 2 对二者给出显式判据，要求模型按查询措辞选择参照系；规则 3 要求"物体的某一部分"（车灯、标志、手、后视镜、轮子）只框该局部而非整物；规则 5 把颜色词当作硬约束，禁止用位置"看似正确"但颜色不符的物体顶替。

几何侧把方位词映射为 (轴, 方向)：left/top → +1、right/bottom → -1（与 detect_select.py 的 _DIR2AXIS 同口径），与序数 rank 组合后即可在候选集上做确定性选择（§3.3.2）；closest/farthest 则映射到深度轴，由深度图 ROI 平均深度排序。

3.3.4 深度与红外的辅助判别

三模态并非简单叠加，各有明确的判别职责：

- **深度 → 距离与前后相对位置**：深度图（16 位、单位毫米）经量程外离群剔除后线性归一化，并给出**有效区掩码**与**四级置信度**（high/medium/low/skip，按有效像素占比划分，见 preprocess.depth_confidence）——本题数据中深度无效区占比偏高（实测中位 25.18%），故掩码是必备而非可选。分图输入时深度以"近处暗、远处亮、无效区为黑"的伪 RGB 呈现，模型据此判断遮挡与前后关系；规则侧则可用**框内 ROI 有效像素的平均深度**对候选排序，closest/nearest/farthest/furthest 远近类查询即按此取框（matcher.match_query_with_depth；detect_select.select_box 为同思路的显式版本）。深度缺失或置信度不足时**宁可回退常规分支也不猜**。
- **红外 → 热辐射与弱光补充**：红外像素值越高表示温度越高，可在可见光失效（夜间、逆光、遮挡）时突显活体与热源轮廓。工程内先转灰度并中值滤波去噪，再以「红外热像」角色作为独立模态输入，使模型在可见光证据不足时有**第二证据源**；低光时亦可用于加权增强可见光（§3.2 的低光融合路径）。
- **可见光 → 外观与纹理**：作为主模态承担类别、颜色、部件等细粒度判据（对应提示词规则 3 与规则 5），最终 bbox 也定义在可见光坐标系上。

三者共同支撑"相对于某物"的方位与序数判断：判断某人相对某车的左右与前后时，可见光负责定位人与车并确认为同类目标，红外用于确认活体、避免与车体阴影混淆，深度用于分离人与车的前后关系，再由 §3.3.3 的参照系规则决定

左右取向。**温度本身不做数值估计**，仅以红外强度间接反映。

3.4 后处理与提交格式（src/postprocess/、make_sub.py）

- **坐标规范**：bbox 归一化到 [0,1]，坐标 round 至 4 位小数；
- **异常回退**：非法或缺失的预测（反向坐标、越界、NaN、空框）回退到占位框 [0.49, 0.49, 0.51, 0.51]；
- **字段回显**：每查询条目附原始三模态路径与查询文本，除 bbox 外的四个字段逐字回显，不改动任何字符；
- **打包**：形成 submission.json 并 zip 压缩（内部文件名 submission.json）供平台提交。

3.5 多产物共识投票后处理（scripts/consensus_vote.py）

3.5.1 问题

零样本生成式定位的单次解码输出（下称**主件**）存在两类不稳定性：

1. **解码波动**：bfloat16 浮点与采样噪声使同一配置的两次全量推理在 11% 的条目上给出不同框；
2. **系统性偏差**：在部分困难位（目标遮挡、同类物并存、序数指代歧义）上，主件与其它解码结果的判断不一致，而单一模型无法自证孰是孰非。

换言之，主件在"与其它解码结果不一致的位置"上并无额外可信度。这正是后处理要处理的靶区。

3.5.2 方法

引入**多产物共识投票**，把"该采信哪个框"转化为"哪个框获得更一致的第三方支持"。

记主件框 b_main、参照件框 b_ref（另一次独立解码结果）、投票池 P（|P| = 32 套）。对每个查询位：

第一步：候选面筛选。 仅当 IoU(b_main, b_ref) < 0.5（**真分歧位**，即两套结果指向不同区域）时进入投票判定；IoU ≥ 0.5 的同侧位两套结果本就一致，改动不构成有效修正，直接跳过。该步骤将判定范围从 5690 条收缩到真分歧位，避免在无分歧处做无意义扰动。

第二步：逐成员表态。 对池中每个成员 p（属于池 P），比较 IoU(b_p, b_main) 与 IoU(b_p, b_ref)：

- 若前者更大 → 记主件票 sa += 1；
- 若后者更大 → 记参照件票 sq += 1；
- 若两者相等 → **弃权**，不计票。

第三步：判决。 当且仅当 sa == 0 且 sq == |P|（主件零票、参照件满票）时，采纳参照件框；其余候选位一律保持主件框。

对应实现（scripts/consensus_vote.py 核心片段）

```
# scripts/consensus_vote.py —— 核心片段：IoU、投票判决、确定性打包
def _iou(a, b) -> float:
    """两个归一化框 [x1,y1,x2,y2] 的交并比。"""
    ix1, iy1 = max(a[0], b[0]), max(a[1], b[1])
    ix2, iy2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0.0, ix2 - ix1) * max(0.0, iy2 - iy1)
    if inter <= 0:
        return 0.0
    ar = (a[2] - a[0]) * (a[3] - a[1])
    br = (b[2] - b[0]) * (b[3] - b[1])
    d = ar + br - inter
    return inter / d if d > 0 else 0.0

# …（略）_bboxes()：读预测 JSON，取 qid 到 bbox 的映射
# …（略）路径常量与命令行参数解析，见文件头

def main() -> int:
```

```
    cur = json.loads(MAIN.read_text("utf-8"))
    main_bb = _bboxes(MAIN)
    ref_bb = _bboxes(REF)
    with gzip.open(POOL_PATH, "rt", encoding="utf-8") as fh:
        pool = json.load(fh)
    m = len(pool)

    out = {k: dict(v) for k, v in cur.items()}
    n_adopted = 0
    for k in main_bb:
        if k not in ref_bb:
            continue
        # 规则 1：仅真分歧位进入投票判定
        if _iou(main_bb[k], ref_bb[k]) >= 0.5:
            continue
        sa = sq = 0
        for bboxes in pool.values():
            b = bboxes.get(k)
            if b is None:
                continue
            ia, iq = _iou(b, main_bb[k]), _iou(b, ref_bb[k])
            if ia > iq:
                sa += 1
            elif iq > ia:
                sq += 1
        # 规则 2：主件零票、参照件满票 → 采纳参照件
        if sa == 0 and sq == m:
            out[k]["bbox"] = list(ref_bb[k])
            n_adopted += 1

    # 格式自检
    for k, e in out.items():
        b = e["bbox"]
        assert len(b) == 4 and all(isinstance(x, (int, float)) for x in b), k
        assert 0 <= b[0] < b[2] <= 1 and 0 <= b[1] < b[3] <= 1, (k, b)

    OUT_JSON.write_text(json.dumps(out, ensure_ascii=False, indent=2), "utf-8")
    # 固定 zip 内部时间戳 → 同输入必得同 SHA256
    zi = zipfile.ZipInfo("submission.json", date_time=(2026, 10, 5, 0, 0, 0))
    zi.compress_type = zipfile.ZIP_DEFLATED
    with zipfile.ZipFile(OUT_ZIP, "w", zipfile.ZIP_DEFLATED) as zf:
        zf.writestr(zi, OUT_JSON.read_text("utf-8"))
```

3.5.3 判据设计说明

- **以 IoU 作为表态依据**：IoU 是与真值无关的纯几何量，不含任何可调参数、学习权重或经验系数。在无真值的评测环境下，引入任何需要拟合的参数都会带来无法验证的过拟合风险；
- **阈值取满票而非多数票**：满票要求全体成员一致。只要出现任何一张异见票，即判定该分歧存在非单侧成因，保持主件不动。该设计使改动面收缩到一致性最高的一档——即以"牺牲改动数量"换取"改动质量"。这一取向在无真值可校验的场景下是必要的保守选择；
- **弃权即不计票**：当成员框与主件、参照件的 IoU 相等（典型情形是三者互不相交、IoU 均为 0）时不予表态。弃权会降低 sq，使该位无法达成满票——即"证据不足则不动"；
- **计算复杂度**：O(候选位数 × |P|)，本例约 3 万次 IoU 计算，秒级完成，不构成运行瓶颈。

3.5.4 与赛题条款的对应

赛题 §十（四）「推理预测与结果优化」明确鼓励："可采用多模型集成推理（如不同预训练模型预测结果加权平均）提升定位准确性"。

本方法属于该范畴的**离散表决**变体：不做加权平均（以免引入权重这一在无真值下无法校验的自由参数），而以"全体一致"作为唯一的采纳条件。投票池由同一推理工程的多组独立运行输出汇总而成，随包分发以保证离线可复现。

3.5.5 确定性保证

管线固定 zip 内部时间戳，投票规则不含任何随机性，因此**同输入必得同 SHA256**，结果可由第三方逐条复算核对。

3.6 提交件校验与结果

产出 result.zip（内部 submission.json，共 5690 条）后执行四项硬约束校验，全部通过：

校验项	判据	结果
条数	与测试集一致（5690 条）	通过
键集	与 queries.json 完全一致	通过
字段回显	visible/infrared/depth/query 四字段逐字一致	通过
bbox 合法性	归一化、x1 < x2、y1 < y2 且落在 [0,1]	通过

平台评测指标 **ACC@0.5 = 0.8116**。

4. 运行配置（configs/server_32b.yaml 关键项）

```yaml
data:
  root: "/root/autodl-tmp/work"            # 测试集根目录（Images/、queries.json）
model:
  type: "qwen3_vl"
  name_or_path: "/root/autodl-tmp/models/Qwen3-8-27B"
  device: "cuda:0"
  use_half_precision: true
  box_threshold: 0.20
  text_threshold: 0.15
inference:
  max_queries_per_prompt: 20
  enable_multimodal_fusion: true
  low_light_threshold: 60
  fusion_mode: "low_light"
  multimodal_input_mode: "split"
  enable_depth_prior: false
  chain_enforce: "off"
  diag_dump: false
qwen:
  output_mode: "multi_box"
  max_new_tokens: 1024
  enable_thinking: false
  feature_cache: true
  cache_max_entries: 4
postprocess:
  fallback_bbox: [0.49, 0.49, 0.51, 0.51]
```

5. 环境与复现

依赖	版本/要求
GPU	显存 ≥ 60GB（实测 RTX PRO 6000 Blackwell 96GB）
系统	Linux（Ubuntu），磁盘 ≥ 100GB
Python	3.12
CUDA	12.4

关键依赖：torch≥2.6、transformers≥5.18.0、accelerate、opencv-python、Pillow、numpy、PyYAML、tqdm（详见 requirements.txt）。

模型权重与数据集未随包分发，获取方式见 §6。

复现步骤（完整版见 code27b/README.md）：

```
pip install -r requirements.txt
# 下载模型：modelscope snapshot_download('Qwen/Qwen3.8-27B', cache_dir='./models')
# 放置测试集：data.root 下 Images/visible|infrared|depth 与 queries.json
python scripts/infer.py --config configs/server_32b.yaml
python scripts/consensus_vote.py                    # 共识投票后处理，产出 result.zip
```

6. 大文件与外部链接

本方案的作品文档（技术方案 PDF、代码与数据包）及其链接**不包含模型权重、数据集、运行环境等大文件**：随包材料全部为代码、配置与说明，合计约 7.4 MB。大文件按下方**独立链接**获取：

大文件	体积	获取方式（独立链接）
模型权重 Qwen3.8-27B-Instruct	约 54 GB	ModelScope 官方地址 Qwen/Qwen3.8-27B；另附 download_model.py 与 18 分片 SHA256 校验
赛题测试数据	—	由赛事方提供，未随包分发
运行环境	—	依赖清单见 code27b/requirements.txt（未提供镜像）

权重下载与逐分片校验：

```
pip install modelscope
python download_model.py                # 下载到 ./models/Qwen3.8-27B
python download_model.py --verify       # 对照 18 分片 SHA256
```

分片哈希清单见 AIC-2026-14375345-模型文件/model_sha256.txt；测试数据按configs/server_32b.yaml 的 data.root 放置，不入包。